Same Rewrite, Different Sign: Attributing and Guarding Hardware-Dependent Graph Rewrites in a Production ML Compiler

Anonymous Authors
Paper #____

Abstract—TorchInductor applies a few hundred graph-rewrite rules whenever they match, with no check that the rewrite is faster on the target. We make every rule, flag and opt-in pass of TorchInductor individually switchable from outside the compiler, calibrate a timing oracle on 480 null runs (false alarms 9% for one comparison, 1.5% with two time-separated confirmations), and attribute a measured slowdown to the responsible rule or pair by delta debugging over the switch set. On 24 models and six machine/precision targets (Apple M4 Pro, Intel Xeon, NVIDIA A40 in fp32 and bf16, L4, RTX 4090) eight of the sixteen most impactful switches have the opposite dominant sign on different machines, while the structural change the compiler makes is identical; attribution recovers planted culprits in 5–16 oracle calls and finds genuine pair interactions, and an uncalibrated oracle fabricates them. Acting on the blame, learned per-rule guards remove the harm where it is frequent and shape-predictable (CPU matmul decomposition from 261% to 1% lost per firing; MobileNet 29% and EfficientNet 41% faster) and, where a rule is right most of the time, the measurements fix a default: freezing, off by default, is faster on 105–113 of 163 real networks on each GPU (median 7–10%), a finding we filed upstream together with a Dynamo bug in the setting that would remove its memory cost.

I. Introduction

torch.compile lowers a PyTorch model through TorchInductor, and a large part of what Inductor does is graph rewriting: a few hundred rules each look for a pattern (a manual attention computation, a convolution followed by batch normalisation, a matrix multiplication of a particular shape) and replace it with something the rule's author expected to be faster. Rules fire whenever they match. Nothing measures whether the result is faster on the target, and when a compiled model is slower than expected there is no way, for users or for the compiler's developers, to find out which rule did it [1]. The only published per-optimization view of Inductor is an eight-knob ablation on one GPU reported as a geometric mean [2].

This paper makes the question answerable and reports what the answer looks like on six machine/precision targets. Our thesis is that a graph rewrite fixes the shape of the generated program, and whether that shape is faster depends on the relative cost of library calls, generated kernels and memory traffic on the target, which no rule written once can know. Three measured facts support it: half of the most impactful rules are confirmed to help on one machine and hurt on another (§IV); when they flip, the compiler has made the identical structural change on both machines (Table II); and acting on the measurement pays, either through a learned guard per rule where harm is frequent and predictable or through a per-machine default where it is not (§V).

Contributions.

1) RuleScope: per-rule switches for every pattern rule, configuration flag and opt-in pass of TorchInductor 2.14 (269/396/398 switches on the Apple, x86 and CUDA builds), installed from outside the compiler by wrapping the hook it already calls before applying a rule, with fire and suppression counts (§II).
2) A timing oracle whose false-alarm rate was measured on 480 runs of unchanged configurations and reduced from 9% to 1.5% by two time-separated confirmations, and delta-debugging attribution over switch sets that recovers planted culprits in 5–16 oracle calls, detects pairs, and declines to blame when nothing is slower (§III).
3) A study of 24 models on an Apple M4 Pro, an Intel Xeon, and NVIDIA A40, L4 and RTX 4090 GPUs in fp32 and bf16: the rule×machine sign matrix, its mechanism, and the effect of precision on which rules fire at all (§IV).
4) Two ways to act on blame, both evaluated on held-out models against a per-model oracle: learned per-rule guards, and per-machine defaults; the latter yields a concrete upstream finding about freezing backed by 163 real networks per GPU, and a Dynamo bug (§V).

II. Per-Rule Switches

Inductor applies rewrites in pre-grad, joint-graph and post-grad passes; most are PatternMatcherPass instances holding PatternEntry rules keyed by the operator they match, and a pass applies a rule only if entry.extra_check(match) returns true. RuleScope replaces every entry's extra_check with a gate that refuses the

match when the rule's switch is off, counts a fire when the original check accepts and the switch is on, and counts a suppression when the rule would have fired but was disabled. No PyTorch source is modified; Inductor's own in-tree learned heuristics use the same hook from the inside [3]. Rules register lazily, so discovery compiles a tiny warm-up graph (attention, conv/batch-norm, split/cat, matmul, a dtype round trip) and then walks the fx_passes modules; direct calls to the initialisers register patterns twice. Configuration flags (freezing, layout_optimization, max_fusion_size, …), opt-in passes enabled through the fusion-option dictionaries, and master switches for whole pass groups complete the universe. A state is the set of switches that are on; because universes differ per platform (oneDNN rules exist only on x86, half-precision attention variants only with CUDA) states are compared by their difference from the default. The FX-graph cache would otherwise return a program compiled under another state, so a custom pass whose uuid() hashes the active state keeps the cache key honest, and the cache is disabled while timing.

Most rules never match a given model: a verify step flips each switch alone from the default, compiles, and hashes the normalised generated code; for BERT about ten of 398 switches change the program. Everything downstream works on that short list.

III. A Calibrated Oracle and Delta-Debugging Attribution

Measurement. One measurement compiles a model under a state in a fresh subprocess, checks the output against eager mode, records the generated code (kernel count, external calls, allocation, a program hash) and times 10 warm-up and 50 timed calls with the stream synchronised on CUDA; results are stored keyed by model, machine, state and session, so every number here is regenerable from the stores.

Calibrating the oracle. Within a run the call times are right-skewed; between runs the spread is fat-tailed. We measured 480 runs of unchanged configurations and asked ten decision rules how often they call one of them "slower": standard deviations, interquartile range, Wasserstein distance, hierarchical bootstrap, Mann–Whitney on pooled calls, and MAD bands. Every single-comparison rule was wrong 6–16% of the time (Fig. 1, right). The adopted oracle takes the median of each run, seven runs in separate processes for a noise model with centre c and $\tau = \max(3\,\mathrm{MAD}, 1\%\,c)$, and calls a state slower only if two time-separated runs both exceed $c+\tau$ with a fresh baseline run in between (8.9% false alarms for one run, 1.5% for two, 0.9% for three). Byte-identical generated programs are never blamed. The min statistic has fewer single-run false alarms on every machine and agrees with the median on 85–91% of sweep verdicts, the rest sitting at the threshold; core pinning helps only sub-50 ms models [4], [5], [6].

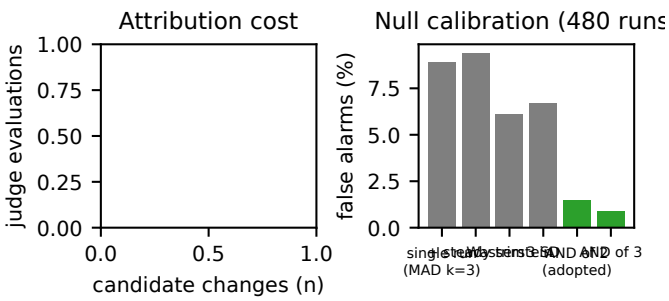


Fig. 1: Left: oracle evaluations per attribution vs. candidate changes (red: pairs); dashed $2\log_2 n + 2$. Right: false alarms of single-comparison rules on 480 null runs vs. the adopted AND rule.

Attribution. Given fast state F and slow state S, the difference is a set of signed changes. Delta debugging [7] over that set with the oracle above returns a 1-minimal set; when it has two or more elements each is tested alone and the result is labelled an interaction if none reproduces the slowdown. Iterative attribution moves the found cause into F and repeats until the residual is within τ; a τ-scan at $k \in \{2, 3, 4, 6\}$ reports stability. The output is the culprit set, its delta, and the kernel-level diff between F and $F \cup C$.

Cases. On the Mac, attention rules 11 and 12 match the same graph: disabling either changes nothing, both costs 34%, and RuleScope reports the pair. On the A40, conv_1x1_as_mm is recovered alone from a 25% slowdown hidden among seventeen opt-in passes in nine evaluations (24→27 kernels, 18→63 MB). On the L4 in bf16 a genuine interaction between two whole pass groups (+6.3%) and the fusion limit (+14.2%) explain an attention block; on BERT the half-precision attention rule (+7.5%) and the fusion limit (+19.0%) explain the whole 0.85 ms gap in 13 evaluations. When the CPU cases were replayed on the GPU the "slow" states were not slower there, and the tool said so. Control. Three cases re-run in one session with the calibrated oracle and an uncalibrated one (one measurement per verdict, no program guard, same first measurements): identical culprits on the two large effects, but where the slow state was within noise the uncalibrated oracle fabricated a two-rule "interaction" and spent five times the evaluations (§VI).

IV. The Same Rewrite on Six Targets

Fig. 2 and Table I give, for the sixteen most impactful switches, on how many corpus models having the switch ON is confirmed to hurt or help on each target. Eight of sixteen have the opposite dominant sign on different machines. Layout optimisation hurts on the Mac (MobileNet 41%, EfficientNet 68%) and the A40 in fp32 (ResNet-18 9%) and helps on the Xeon (up to 90%) and every bf16 GPU; the fusion-size limit helps everywhere but the Xeon;

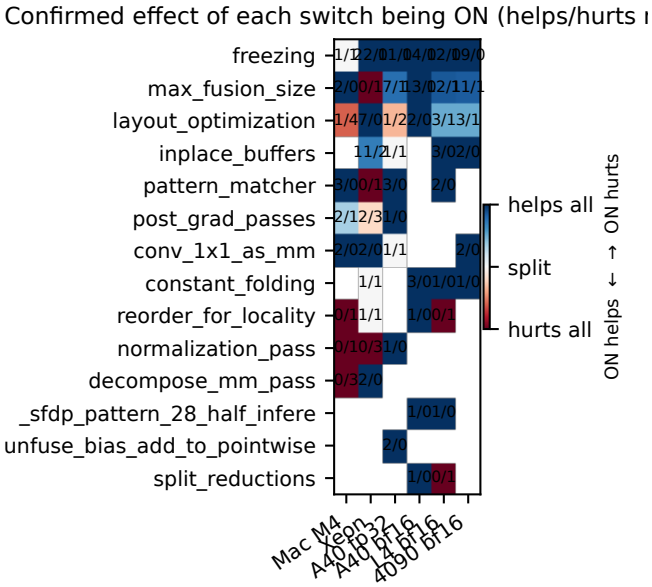


Fig. 2: Rule×machine sign matrix: colour is the balance of models helped vs. hurt by the switch being ON; cells give helped/hurt counts (confirmed, ≥ 3%).

matmul decomposition costs 135% on the Mac and saves 95% on the Xeon; freezing helps wherever it does anything.

Mechanism. Joining each confirmed verdict with the code-level change the same switch makes (Table II) shows the compiler making the same structural change on every machine. decompose_mm removes 96–106 external BLAS calls on both CPUs: 39% slower on the Mac, whose BLAS handles small matmuls well and whose generated loops do not, 53% faster on the Xeon. Layout optimisation adds two kernels and 4–22 MB of allocation on both CPUs because the batch-norm/ReLU epilogues stop fusing into the convolution: −28% on the Mac, +14% on the Xeon whose oneDNN convolutions prefer channels-last. The fusion limit removes 8–12 kernels on the GPUs and is worth 7–12% on 11–13 models per GPU. Relay noted in a sentence that pass effectiveness is "model- and device-dependent" [8]; this is the per-rule, six-target version of that sentence.

Precision changes the rule set. In fp32 no attention-fusion rule fired on any GPU: a GPU-only softmax rewrite fires first and the fusion pattern no longer matches. In bf16 the half-precision variant of rule 28 fires on BERT and is worth 7.5% on the L4. A GPU study in fp32 would have concluded that attention fusion never fires.

V. Acting on the Blame

Learned guards. A guard is a small model in the same extra_check hook that decides whether to let a rule fire. Rows are (features at the firing, measured effect of blocking it), from synthetic sweeps (attention B/T/H/D, conv C/R/K/L/depthwise, decode-MLP M/I/Hd; 1,516 rows on the CPUs) and from real networks (38 torchvision conv nets, 14 transformer architectures; 163 rows per switch per GPU, Table V). Features are the shapes at the match for pattern rules and a whole-model profile (conv work and its depthwise/1×1 fractions, linear work and how decode-shaped it is, attention shapes, parameters, batch, tokens) for model-level decisions. Trees, gradient-boosted trees, logistic regression, a fine-tuned encoder and a commercial decision API are compared on grouped splits that hold out whole shape slices or architectures; the score is run time lost per firing against "always fire". On the CPUs the guards lose 1.0–1.4% per firing where always-fire loses 262% (decode MLP) and 176% (layout); end to end the layout guard makes MobileNet 29% and EfficientNet 41% faster on the Mac with no regression, and the matmul guard with the decomposition pass enabled makes SmolLM decode 58% (Mac) and GPT-2 decode 73% (Xeon) faster (Table IV). On the A40 in fp32 a layout guard trained on 60 real networks made held-out ResNet-18 8% and ResNet-50 4% faster, recovering 97% of the per-model oracle on the conv stack.

Where guards do not beat the default. A guard trained on 1–3-layer synthetic stacks did not transfer to real GPU networks until trained on real networks: the mechanism was fine, the distribution was not. On the bf16 GPUs the layout rule hurts a minority of real networks (18–21 of 111 with an effect) and helps the majority (62–83), so no learned guard beats "always on" in cross-validation there. Guards pay where harm is frequent and shape-predictable; otherwise the measured answer is a per-machine default.

A default: freezing. It folds parameters into constants and is off by default. Across 163 real networks on each of three GPUs it is confirmed faster on 105–113 and slower on at most one, median 7–10%, p90 18–24%, up to 65% (Table VI, Fig. 3); it helps 22 of 26 corpus models on the Xeon, up to 95%, and is a wash on the Mac, which has no oneDNN path. Verdicts and magnitudes are the same with published weights as with random initialisation (Table VII, 10–12 of 12 agree per machine). SmolLM-135M generate with a KV cache is 8–22% faster on an A5000 with identical output; ResNet-50 classification 4% (GPU) and 11% (Xeon). The costs are compile time (+10–30%) and peak memory (+45–51% on decoder models). freezing_discard_parameters removes the duplicate (ResNet-50 121→75 MB, BERT 280→235) at no speed cost but crashes Dynamo on every tied-embedding language model, which we reduced to ten lines on the CPU. Both were filed upstream (link withheld for review).

VI. Repeatability, Controls and Threats

Same-machine verdicts repeat: 213 of 220 single-switch verdicts agree across sessions on the CPUs and 188 of 221 on the A40; the disagreements are 2–3% effects at the threshold and sub-millisecond toys. Across machines 42 of 47 disagree, which is the finding of §IV. The control experiment of §III shows what an uncalibrated oracle

switch (ON vs OFF)	Mac M4 (24 models)	Xeon (26 models)	A40 fp32 (23 models)	A40 bf16 (21 models)	L4 bf16 (21 models)	4090 bf16 (21 models)
freezing	hurts 1, helps 1 (7%)	helps 22 (95%)	helps 11 (26%)	helps 14 (39%)	helps 12 (37%)	helps 19 (49%)
max_fusion_size	helps 2 (7%)	hurts 1 (5%)	hurts 1, helps 7 (9%)	helps 13 (15%)	hurts 1, helps 12 (15%)	hurts 1, helps 11 (21%)
layout_optimization	hurts 4, helps 1 (40%)	helps 7 (90%)	hurts 2, helps 1 (24%)	helps 2 (22%)	hurts 1, helps 3 (26%)	hurts 1, helps 3 (9%)
inplace_buffers	·	hurts 2, helps 11 (283%)	hurts 1, helps 1 (17%)	·	helps 3 (19%)	helps 2 (22%)
pattern_matcher	helps 3 (34%)	hurts 1 (12%)	helps 3 (28%)	·	helps 2 (7%)	·
post_grad_passes	hurts 1, helps 2 (17%)	hurts 3, helps 2 (283%)	helps 1 (11%)	·	·	·
conv_1x1_as_mm	helps 2 (6%)	helps 2 (20%)	hurts 1, helps 1 (24%)	·	·	helps 2 (9%)
constant_folding	·	hurts 1, helps 1 (10%)	·	helps 3 (25%)	helps 1 (44%)	helps 1 (48%)
normalization_pass	hurts 1 (16%)	hurts 3 (217%)	helps 1 (11%)	·	·	·
decompose_mm_pass	hurts 3 (135%)	helps 2 (95%)	·	·	·	·
reorder_for_locality	hurts 1 (4%)	hurts 1, helps 1 (5%)	·	helps 1 (3%)	hurts 1 (16%)	·
decompose_tanh	helps 2 (6%)	·	·	·	·	·
unfuse_bias_add_to_pointwise	·	·	helps 2 (12%)	·	·	·
_sfdp_pattern_28_half_inference	·	·	·	helps 1 (5%)	helps 1 (8%)	·
split_reductions	·	·	·	helps 1 (4%)	hurts 1 (6%)	·
_sfdp_pattern_28_inference	helps 1 (5%)	·	·	·	·	·

TABLE I: Rule × machine sign matrix. For each switch, the number of corpus models on which having the switch ON is confirmed to hurt or help (two time-separated runs beyond the machine's noise band, effect $\geq 3\%$), with the largest confirmed effect. Eight of the sixteen most impactful switches have the opposite dominant sign on different machines.

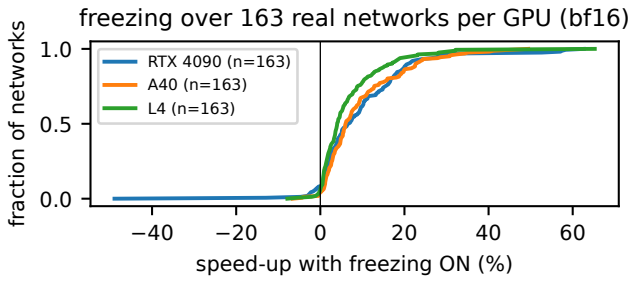


Fig. 3: Speed-up from freezing over 163 real networks per GPU (bf16).

does: on the case whose slow state was within noise it reported a batch_tanh× decompose_mm interaction that does not exist, in eleven evaluations, where the calibrated oracle said "not slower" in two. PyTorch 2.13 registers 267 switches on the Mac against 269 for 2.14 (four rules added); the subset sweep under 2.13 appears as additional columns of Fig. 2 in the final version. Threats: inference only; one compiler version for the full study; random weights for most of the corpus with 12 pretrained twins agreeing; 163-row GPU datasets; a shared VM and rented pods, which is why every verdict needs two confirmations.

VII. Related Work

Delta debugging is Zeller's [7]; applied over compiler passes it localises miscompiles with a Boolean oracle (opt-bisect, BugLens, MultiConf), and performance bisection over commits is unreliable under noise [9], which is why ours is calibrated. Pečimúth et al. localise Graal performance problems by diffing recorded optimisation decisions [10]; we intervene and obtain a counterfactual culprit, closer in spirit to causal profiling [11]. Learned compiler heuristics span Milepost, MLGO [12] and DeepTune [13]; ML compilers learn cost models to drive schedule or rewrite search (Ansor [14], TASO [15], Tensat [16]), and such models are known to be hardware-bound [17]; Inductor's own autoheuristic [3] learns one or two decisions per GPU. That optimisation benefit depends on the target is old news at the flag level [18], [19]; none of these report per-rule sign flips in a production ML compiler, and the only prior per-optimisation ablation of Inductor is Table 4 of the PyTorch 2 paper [2]: eight knobs, one A100, geometric mean, no noise model, measured with freezing on. Benchmarking methodology follows [4], [5], [6], [20].

VIII. Conclusion

A rewrite rule in a production ML compiler is a bet about hardware its author did not have. With per-rule switches, a calibrated oracle and delta debugging the bet can be checked, the losing rule named, and the loss removed: by a guard where it is predictable, by a default where it is not.

References

[1] "Feature: Pattern matcher observability," PyTorch dev-discuss, 2025.

[2] J. Ansel et al., "Pytorch 2: Faster machine learning through dynamic python bytecode transformation and graph compilation," in ASPLOS, 2024.

switch	machine	ON helps: n, median %, Δkernels, Δextern calls, Δalloc MB	ON hurts: n, median %, Δkernels, Δextern calls, Δalloc MB
layout_optimization	Mac M4	1, 10%, +2, +0, +22	4, 28%, +2, +0, +4
layout_optimization	Xeon	7, 14%, +2, +0, +10	·
layout_optimization	4090 bf16	3, 4%, +6, +0, +7	1, 6%, +12, +0, +2
layout_optimization	L4 bf16	3, 19%, +6, +0, +7	1, 4%, +18, +0, +1
layout_optimization	A40 bf16	2, 21%, +8, +0, +9	·
freezing	Mac M4	1, 6%, +0, +0, -0	1, 7%, -10, +0, -12
freezing	Xeon	22, 8%, +0, +0, -18	·
freezing	4090 bf16	19, 11%, +0, -1, +0	·
freezing	L4 bf16	12, 11%, -0, -0, -0	·
freezing	A40 bf16	14, 13%, +0, -18, +0	·
max_fusion_size	Mac M4	2, 5%, +0, +0, +0	·
max_fusion_size	Xeon	·	1, 5%, +0, +0, +0
max_fusion_size	4090 bf16	11, 12%, -8, +0, -0	1, 21%, -11, +0, +0
max_fusion_size	L4 bf16	12, 7%, -12, +0, -0	1, 15%, -11, +0, +0
max_fusion_size	A40 bf16	13, 8%, -8, +0, -0	·
decompose_mm_pass	Mac M4	·	3, 39%, -3, -96, +0
decompose_mm_pass	Xeon	2, 53%, -4, -106, +0	·
pattern_matcher	Mac M4	3, 18%, -2, -1, -7	·
pattern_matcher	Xeon	·	1, 12%, -3, -1, -7
pattern_matcher	L4 bf16	2, 6%, -4, -6, -43	·
post_grad_passes	Mac M4	2, 10%, -0, +0, -0	1, 3%, +2, +0, +0
post_grad_passes	Xeon	2, 144%, +0, +0, -0	3, 27%, +0, +0, +0
inplace_buffers	Xeon	11, 9%, +0, +0, -64	2, 50%, +0, +0, -1
inplace_buffers	4090 bf16	2, 16%, +0, +0, -4	·
inplace_buffers	L4 bf16	3, 8%, +0, +0, -1	·
conv_1x1_as_mm	Mac M4	2, 5%, +0, +0, +36	·
conv_1x1_as_mm	Xeon	2, 16%, +0, +0, +30	·
conv_1x1_as_mm	4090 bf16	2, 9%, +0, +0, +15	·

TABLE II: Mechanism: confirmed verdicts joined with the code-level change the same switch makes (ON minus OFF: kernels, external library calls, static allocation in MB; medians over the affected models). The structural change is the same on every machine; its runtime sign is not.

model	stock ms	τ %	guards: mlp
decode_mlp	0.22	2.6	-11.3% ↓ (3 blocked)
smollm_135m_decode	15.80	4.4	-58.1% ↓ (210 blocked)
qwen2_0_5b_decode	20.98	3.2	-24.5% ↓ (96 blocked)
gpt2_decode	22.27	3.3	-4.1% ↓ (1 blocked)
smollm_135m_prefill	234.28	4.5	-83.2% ↓ (211 blocked)
geometric mean			-46.9%
faster / slower (confirmed)			5 / 0

TABLE IV: End-to-end effect of adding the guards one family at a time (mac mlp); change versus stock Inductor, arrows mark confirmed verdicts beyond τ.

GPU	switch	rows	harmful	helpful	neutral	Δeffect: median (effect) %
4090	constant_folding	163	5	1	27	0.0
4090	freezing	163	1	113	49	0.0
4090	layout_optimization	163	21	62	28	-52.0
4090	inplace_buffers	163	12	30	121	0.0
4090	max_fusion_size	163	12	52	99	0.0
L4	freezing	163	1	105	57	0.0
L4	layout_optimization	163	21	83	7	-52.0
L4	max_fusion_size	163	4	76	83	0.0
A40	conv_1x1_as_mm	163	18	14	55	-76.0
A40	freezing	163	0	112	51	0.0
A40	layout_optimization	163	18	69	24	-53.0
A40	inplace_buffers	163	19	10	134	0.0
A40	max_fusion_size	163	10	71	82	0.0

TABLE V: Real-network switch datasets on three GPUs (bf16): each row is one network measured with the switch on and off, twice when beyond the noise band.

family	guard	accuracy	harmfu
decode MLP (decompose_mm)	Inductor's rule	0.79	
	decision tree	0.93	
	gradient-boosted trees	0.94	
	logistic regression	0.87	
conv layout (channels-last)	Inductor's rule	0.58	
	decision tree	0.80	
	gradient-boosted trees	0.87	
	logistic regression	0.80	
attention fusion (SDPA)	Inductor's rule	0.88	
	decision tree	0.77	
	gradient-boosted trees	0.92	
	logistic regression	0.80	

TABLE III: Guard quality under grouped cross-validation (combined). Cost = mean % of runtime lost per firing by following the guard; always-fire is Inductor's current behaviour.

n	cost %
130	5.0
0	10.3
52	16.5
0	4.8
0	10.8
0	6.7
52	24.0
0	6.6
76	5.8
0	8.4
52	18.6
0	6.9
0	8.4

[3] "Torchinductor autoheuristic (torch/_inductor/autoheuristic)," PyTorch source tree, 2025.

[4] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems," in SC, 2015.

[5] T. Mytkowicz et al., "Producing wrong data without doing anything obviously wrong!" in ASPLOS, 2009.

[6] C. Curtsinger and E. D. Berger, "Stabilizer: Statistically sound performance evaluation," in ASPLOS, 2013.

[7] A. Zeller and R. Hildebrandt, "Simplifying and isolating failure-inducing input," IEEE TSE, 2002.

[8] J. Roesch et al., "Relay: A high-level compiler for deep learning," in MAPL, 2019.

[9] others, "On the effectiveness of bisection in performance regression localization," Empirical Software Engineering, 2022.

[10] A. Pečimúth, D. Leopoldseder, and P. Tůma, "Diagnosing compiler performance by comparing optimization decisions," in MPLR, 2023.

[11] C. Curtsinger and E. D. Berger, "Coz: Finding code that counts with causal profiling," in SOSP, 2015.

GPU	nets	faster	slower	neutral	median gain %	p90	max	conv nets	transformers
RTX 4090 (bf16)	163	113	1	49	10.3	24.2	62.7	71/114	42/49
L4 (bf16)	163	105	1	57	6.7	18.0	69.2	76/114	29/49
A40 (bf16)	163	112	0	51	8.4	24.9	49.4	74/114	38/49

TABLE VI: freezing (off by default) on 163 real networks per GPU: confirmed faster/slower with freezing ON, gains among the faster ones, and the faster fraction by family.

model	random init		trained weights		peak MB off→on	compile s
	gain %	label	gain %	label		
resnet18	+11.6	helpful	+10.9	helpful	48 → 72	3 → 3
resnet50	+11.9	helpful	+11.9	helpful	73 → 121	6 → 8
mobilenet_v3_small	+8.2	helpful	+6.7	helpful	22 → 24	12 → 14
efficientnet_b0	+17.7	helpful	+13.7	helpful	38 → 38	13 → 17
convnext_tiny	+3.1	helpful	+2.9	helpful	80 → 83	5 → 8
bert_base_eagerattn	-3.6	harmful	-0.1	neutral	241 → 280	7 → 10
distilbert	-2.2	neutral	+1.2	neutral	155 → 172	3 → 4
gpt2_prefill	-1.9	neutral	-1.8	neutral	403 → 403	7 → 9
gpt2_decode	+1.4	neutral	+2.5	helpful	253 → 253	6 → 8
smollm_135m_decode	+27.4	helpful	+24.4	helpful	265 → 401	19 → 22
smollm_135m_prefill	+28.2	helpful	+27.9	helpful	301 → 437	17 → 22
qwen2_0_5b_decode	+27.9	helpful	+28.4	helpful	959 → 1406	15 → 19

TABLE VII: freezing on the corpus models with random and with trained weights (RTX A5000, bf16): the verdicts and magnitudes agree; the price is peak memory and compile time.

[12] M. Trofin et al., "Mlgo: a machine learning guided compiler optimizations framework," arXiv:2101.04808, 2021.

[13] C. Cummins, P. Petoumenos, Z. Wang and H. Leather, "End-to-end deep learning of optimization heuristics," in PACT, 2017.

[14] L. Zheng et al., "Ansor: Generating high-performance tensor programs for deep learning," in OSDI, 2020.

[15] Z. Jia et al., "Taso: Optimizing deep learning computation with automatic generation of graph substitutions," in SOSP, 2019.

[16] Y. Yang et al., "Equality saturation for tensor graph superoptimization," in MLSys, 2021.

[17] Y. Zhai et al., "Tlp: A deep learning-based cost model for tensor program tuning," in ASPLOS, 2023.

[18] J. Pallister, S. Hollis, and J. Bennett, "Identifying compiler options to minimise energy consumption for embedded platforms," in The Computer Journal, 2013.

[19] A. Magni, C. Dubach and M. O'Boyle, "Automatic optimization of thread-coarsening for graphics processors," in PACT, 2014.

[20] E. Barrett et al., "Virtual machine warmup blows hot and cold," in OOPSLA, 2017.